1. Introduction

csce585-midi is a highly customizable and modular system for preprocessing, training, generation, and logging for MIDI-based generative models. Rather than generating raw audio, MIDI files are used. MIDI files are symbolic representations of musical events like pitch, velocity, and duration. This enables users to easily manipulate generated output within a Digital Audio Workstation (DAW), allowing them to modify melodies, adjust harmonies, and build on outputs from models in creative or research settings.

To support a wider range of experiments, the system provides two tokenization strategies:

- **Naïve tokenization**, where each note gets represented as a single token. This allows small models to learn easier, but this limits generation to constant 8th note rhythms.
- **Miditok REMI (Revamped MIDI) tokenization** converts each note into several tokens describing pitch, velocity, duration, and other musical features.

csce585-midi is a flexible experimental framework that enables users to:

- Preprocess any MIDI dataset using either tokenization strategy
- Train generator and discriminator models with configurable hyperparameters
- Combine generators, discriminators, and sampling methods during inference (generation)
- Seed generation with any MIDI file
- Logging and evaluation of all generated output with consistent musical metrics

2. Motivation

The project initially began as an attempt to train MIDI-based generative models on the Nottingham dataset, which contains around 1,200 instrumental folk tunes. While the initial goal was pretty narrow, the project eventually evolved into a general-purpose framework for MIDI-based music generation research.

Command-line flags are used by the user to specify things like:

- Preprocessing options
    - Desired dataset
    - Desired tokenization
    - Tokenizing seed files
- Training options
    - Model architecture
    - Preprocessed data to use
    - Hyperparameters
- Generation options
    - Sampling strategies
    - Model(s) to use
    - Seed MIDI file

These allow csce585-midi to function more like a research tool than a fixed pipeline.

My major personal motivation behind this project is my background in Bluegrass music. The Nottingham dataset contains folk tunes from Ireland and England mostly. In America, these songs were passed down over generations and are now played primarily within the genre of Bluegrass.

Bluegrass is very important to me personally. I play in a Bluegrass band with friends and this semester I was the president of the Bluegrass club at UofSC. Due to this (and hardware constraints), I felt compelled to work with the Nottingham dataset for my experiments.

Improvisation is a core component of Bluegrass music. Professional Bluegrass musicians typically embellishes the melody with improvised lines and phrases, which makes their

interpretation of the melody musical and unique. I approached this project with this process of improvisation in mind. I wanted to focus on the melody as a foundation for improvisation rather than a static sequence of notes.

Hardware constraints significantly limited me during the development process. I tested platforms like Google Colab, AWS EC2, and Lambda Cloud. I had the most luck with Lambda, but due to the cost, I was not able to use their GPU instance for very long.

Most of the models were trained locally. These constraints guided the project's design toward lightweight models and efficient preprocessing. Despite this, a user with more resources could preprocess any size dataset and any size model they wish.

3. **Development Process**

I started by replicating the architecture described in “Musenet : Music Generation using Abstractive and Generative Methods”. This architecture uses:

- **A generator** model that predicts the notes that are likely to appear next in a sequence
    - In the paper, they experiment with both LSTM and GPT-2 transformer models for the generator.
- **A discriminator** model trained on notes contained in measures. It generates sets of candidate notes, which are used to help guide the generator.
    - Musenet used a MLP (Multi-Layer Perceptron) model for the discriminator.

The relationship between the generator and discriminator models resembles the way that real musicians improvise. When improvising on a song, musicians consider both the key and the chords they’re playing over. Discriminator models essentially work to prioritize notes that fall within what it believes to be the current key.

Throughout development, the system expanded to include:

- Configurable preprocessing pipelines
- Support for multiple token types
- Modular architectures for generators and discriminators
- Configurable training
- Evaluation tools for analyzing musical metrics

Despite developing this on my MacBook, all components are designed with the intention that with more computing power, the same framework could scale to larger datasets and models. There are instructions on the projects wiki detailing how to enable CUDA.

4. Experiments

I conducted experiments to analyze the effects of:

1. Sampling strategy
2. Discriminator guidance
3. Tokenization method
4. Generator architecture
5. Discriminator architecture

All models were trained on the Nottingham dataset or its augmented version. Musical metrics were computed for each MIDI file generated.

4.1 Baseline Models

Model Name	Model Architecture	Dataset	Epochs	Learning Rate	Embed Size	Hidden Size
Baseline Naïve	LSTM	nottingham_naive	10	0.001	128	256
Baseline Miditok	LSTM	nottingham_miditok	10	0.001	128	256
Augmented Miditok	LSTM	nottingham_miditok_augmented	10	0.001	128	256

The augmented dataset was made by transposing each tune into multiple keys. I anticipated that this would cause less overfitting within miditok models, which had been producing incoherent music in all previous tests prior to these experiments.

4.2 Experimental Metrics

Each generated MIDI file was evaluated for:

- Number of notes
- Duration in seconds
- Note density (notes per second)
- Pitch range (difference between lowest and highest pitch)
- Polyphony (proportion of time where more than 1 note is sounding)

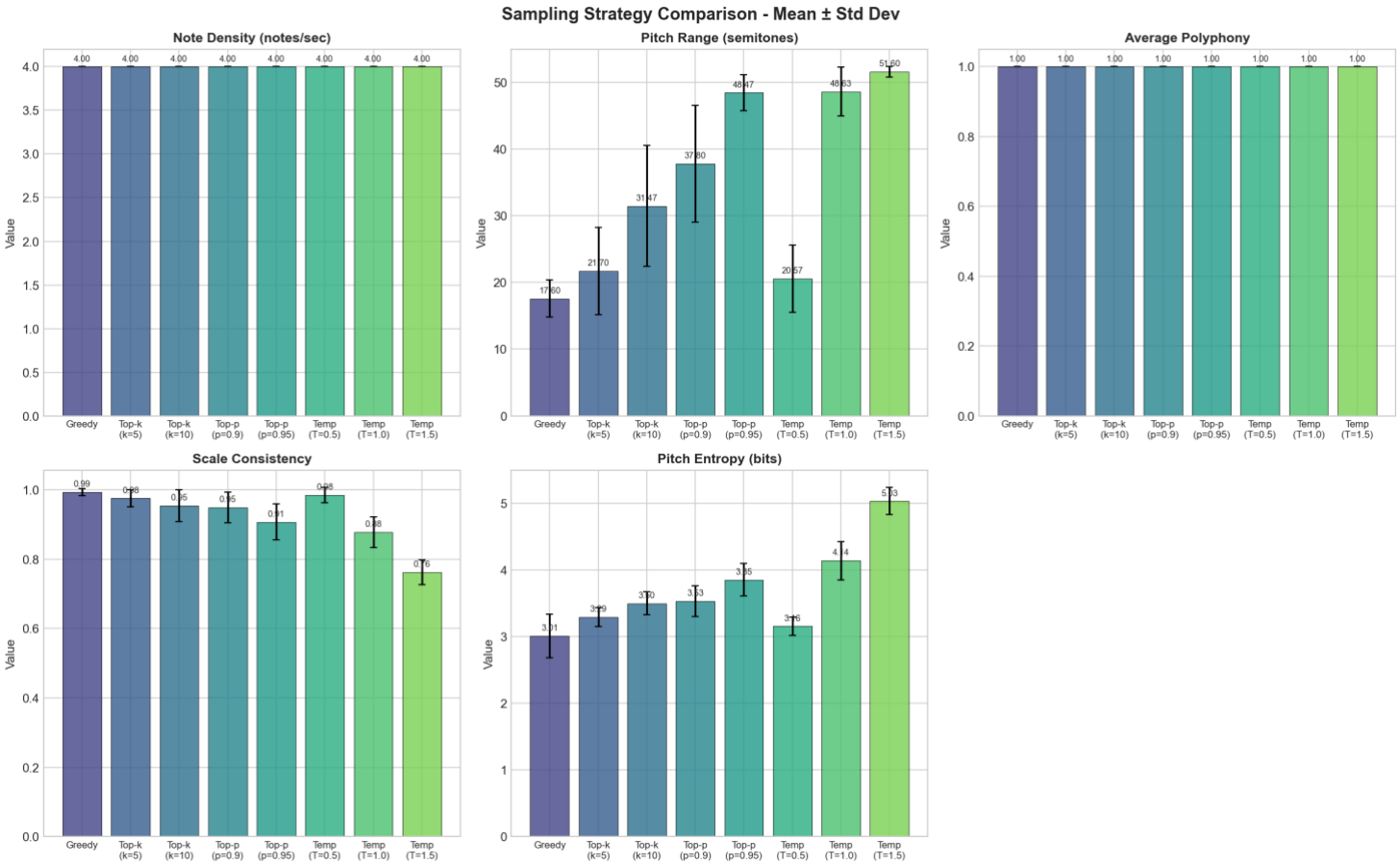
- Scale consistency (fraction of notes which are contained in the major or natural minor scales of another note in the generated sequence)
- Pitch entropy (variation across pitch values)
- Pitch class entropy (variation across pitch classes)
    - Pitch class is the name of the given note regardless of its octave (C3 and C4 are the same pitch class of C)

5. Experiment 1: Effect of Sampling Strategy

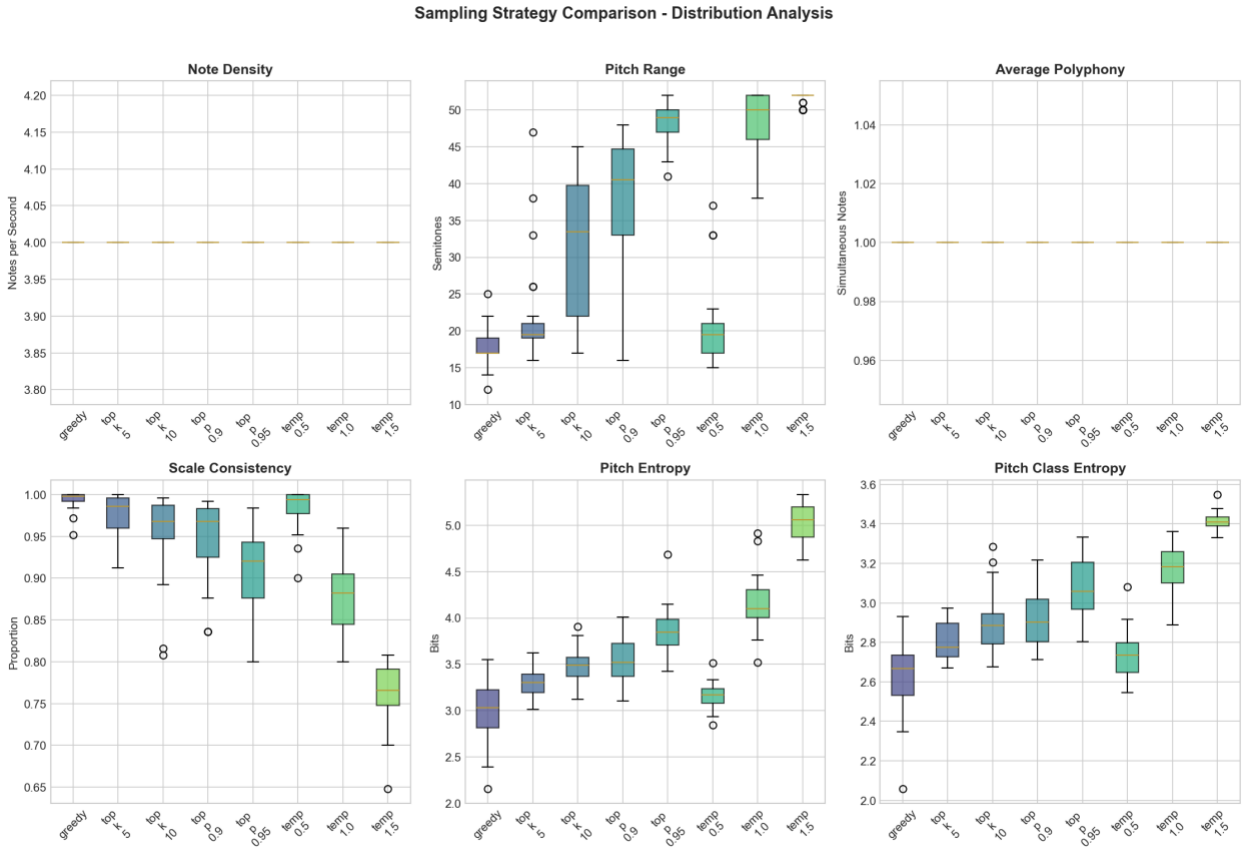
Using the baseline naïve LSTM model, 30 samples were generated for each of these sampling methods:

- Temperature: 0.5, 1.0, 1.5
- Top-k: k = 5, 10
- Top-p: p = 0.9, 0.95
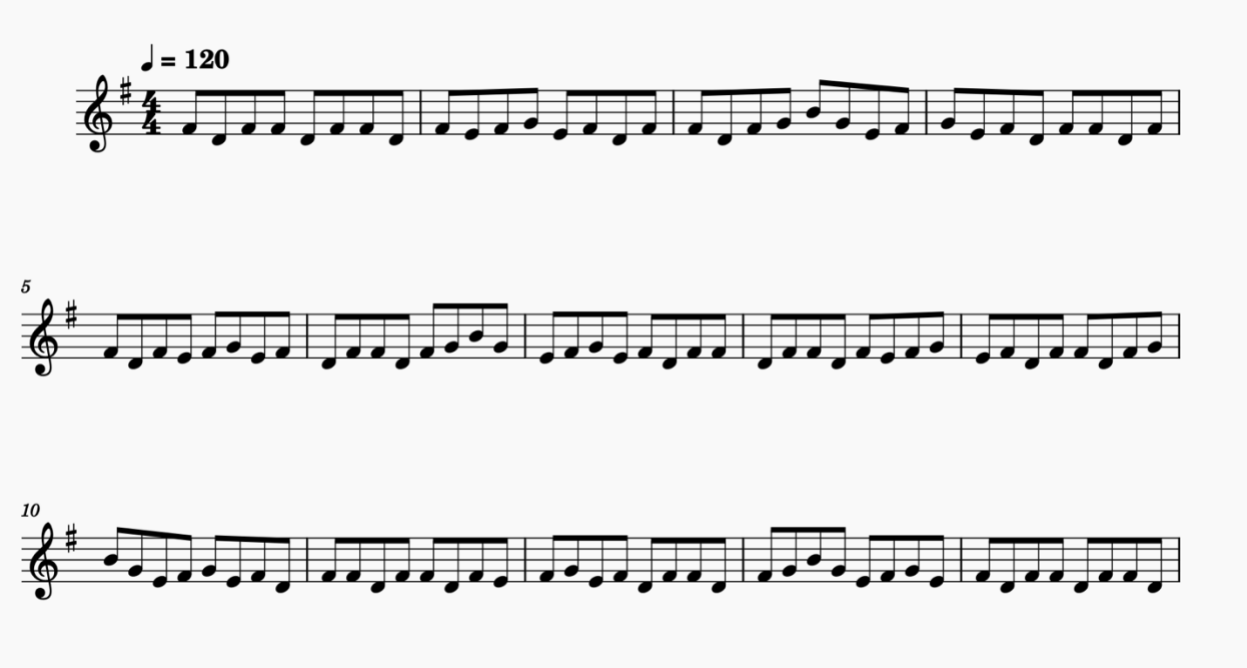
5.1 Sampling Strategy Comparison: Mean +- Standard Deviation
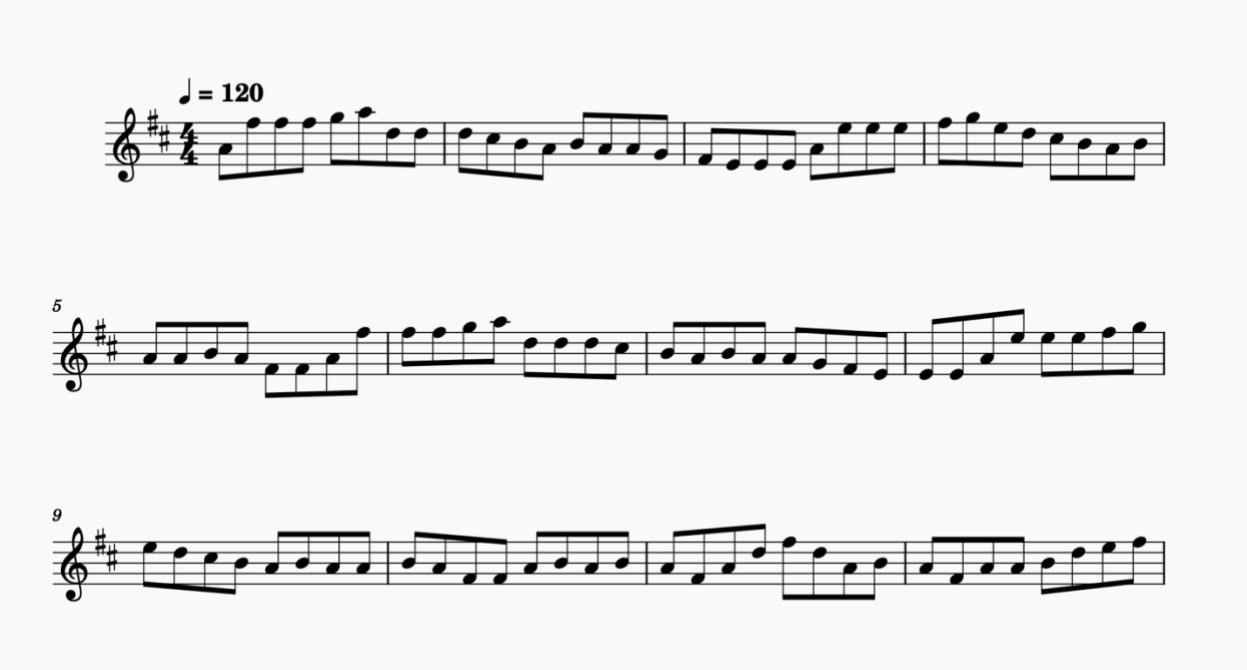


5.2 Sampling Strategy Comparison: Distribution Analysis



5.3 MIDI generated with temperature = 1.0



5.4 MIDI generated with temperature = 2.0



Findings:

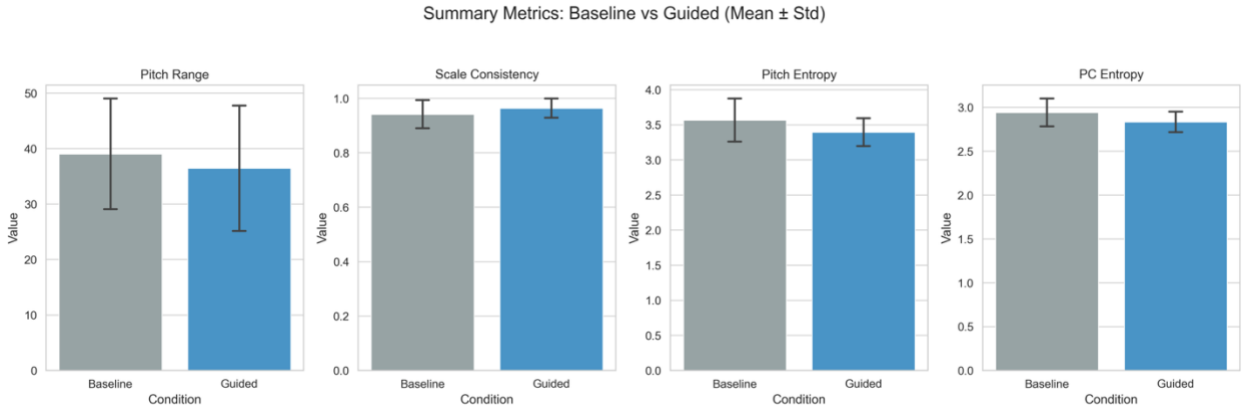
- Higher temperature, k, and p values result in higher pitch variety. (5.3, 5.4)
- Low temperature and greedy sampling frequently collapsed into the same sequence of notes getting repeated constantly.
- Because naïve models only output constant 8th notes, density and polyphony are unchanged across all sampling settings.

These results confirm the sampling behavior I expected and establish a baseline for comparison with midtok models.

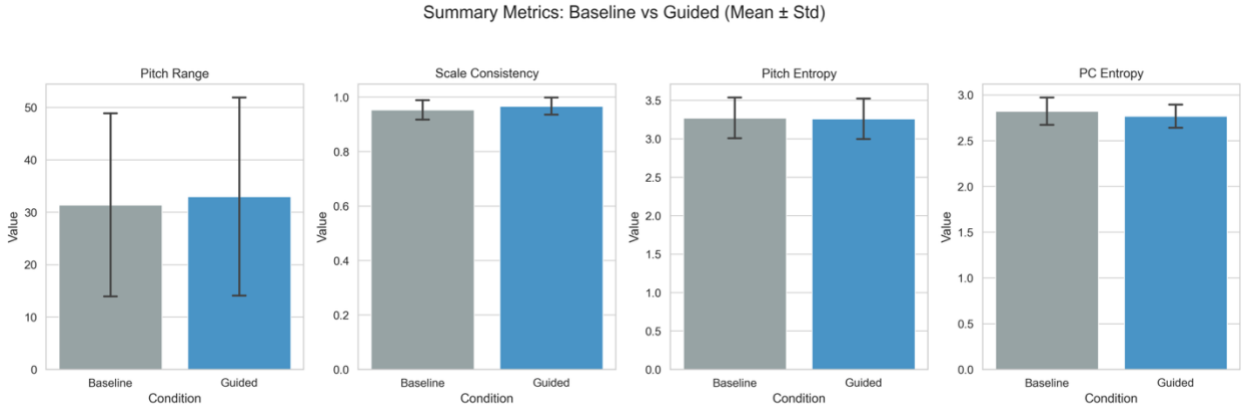
6. Experiment 2: Effect of Discriminator Guidance

To analyze how discriminator guidance alters musical characteristics, 50 samples were generated with and without a trained MLP discriminator (the same model architecture used in Musenet). Experiments were repeated for the naïve and miditok baseline generators.

6.1 Naïve Baseline vs. Guided: Mean+- Standard Deviation



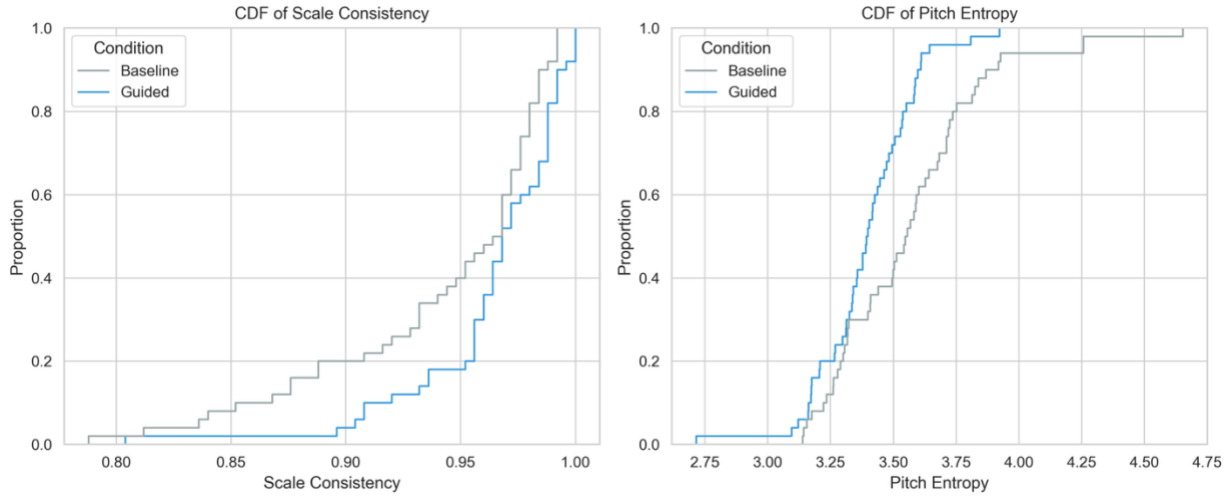
6.2 Miditok Baseline vs. Guided: Mean +- Standard Deviation



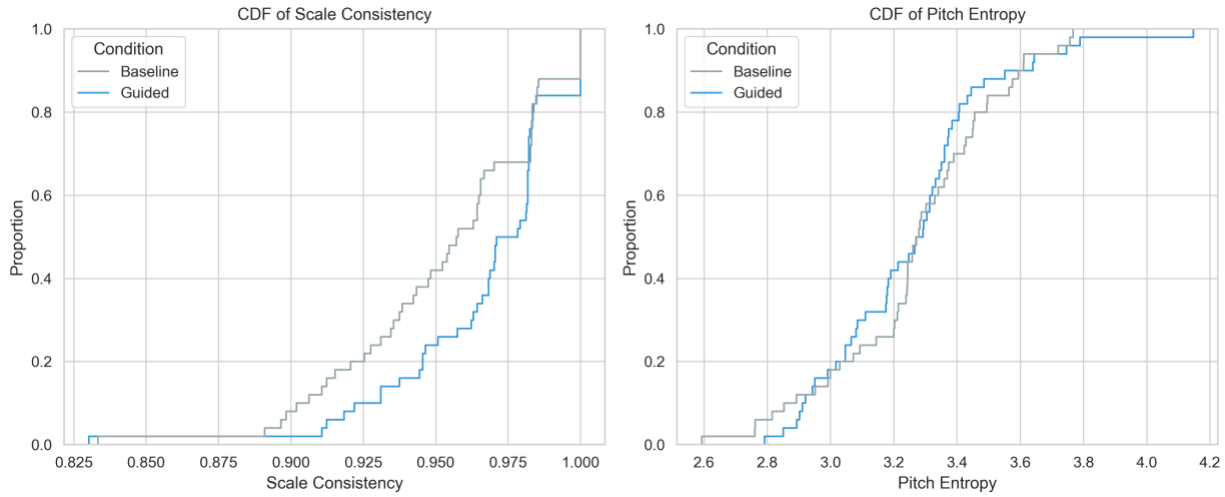
Key results:

- Scale consistency increased for both generators when given discriminator guidance.
- Pitch class entropy increased, which indicates more unique notes being used.
- Miditok models surprisingly increased in pitch range when given guidance. This is likely because guidance prevented it from collapsing into repeated patterns.
- Guided samples sounded more correct but less expressive. They avoided any kind of interesting chromatic note choices.

6.3 Naïve CDF of Scale Consistency and Pitch Entropy

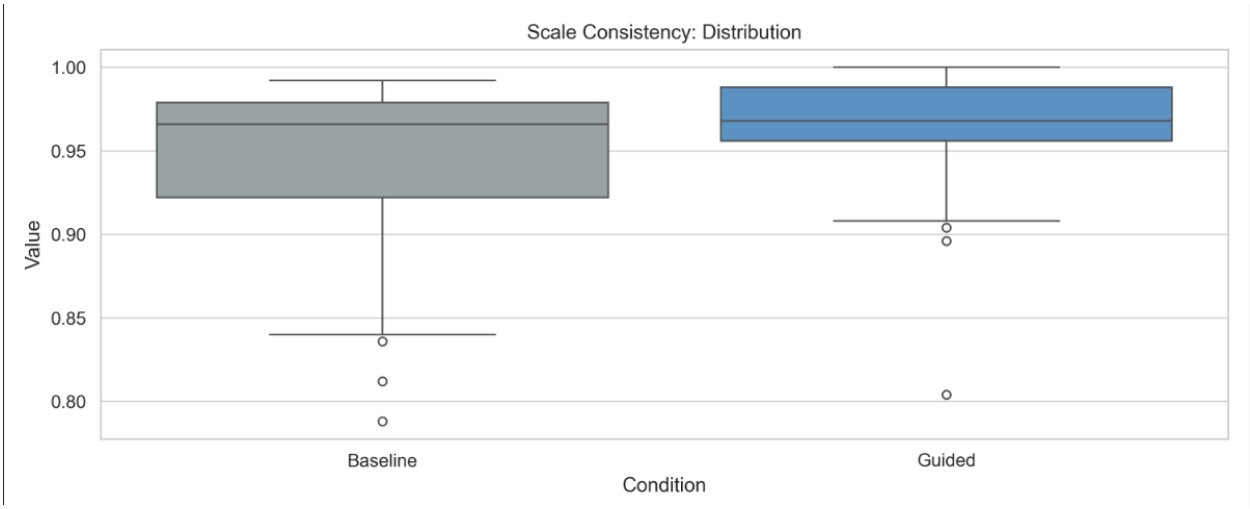


6.4 Miditok CDF of Scale Consistency and Pitch Entropy

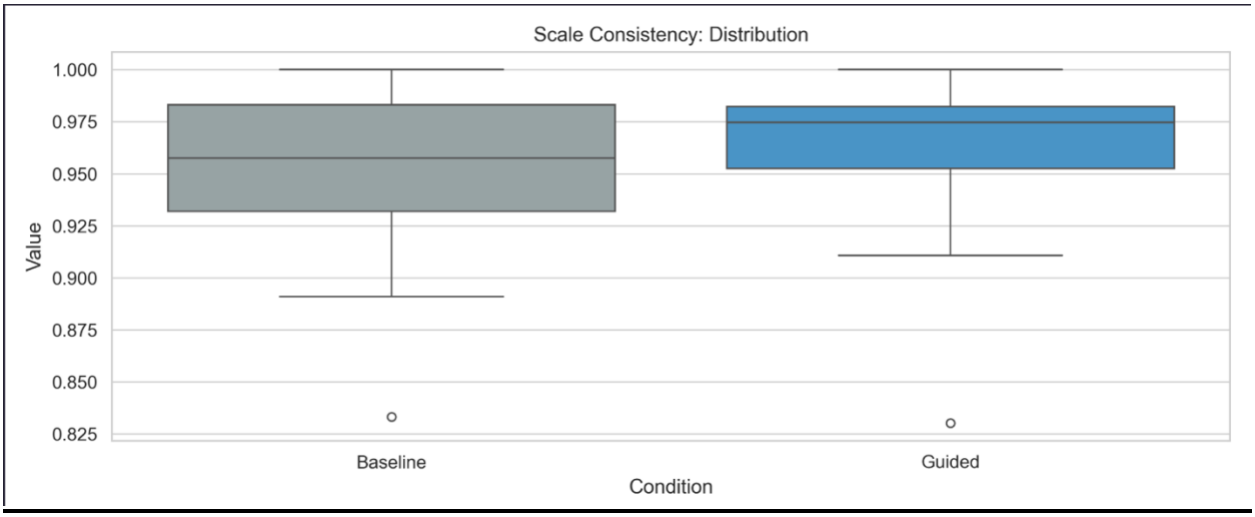


The CDF plots confirm that having discriminator guidance causes higher scale consistency and also reduces pitch entropy.

6.5 Naïve Scale Consistency Distribution



6.6 Miditok Scale Consistency Distribution



Both models experienced an increase in the mean of scale consistency when given guidance from the discriminator model. Both also visibly had smaller ranges in scale consistency as well.

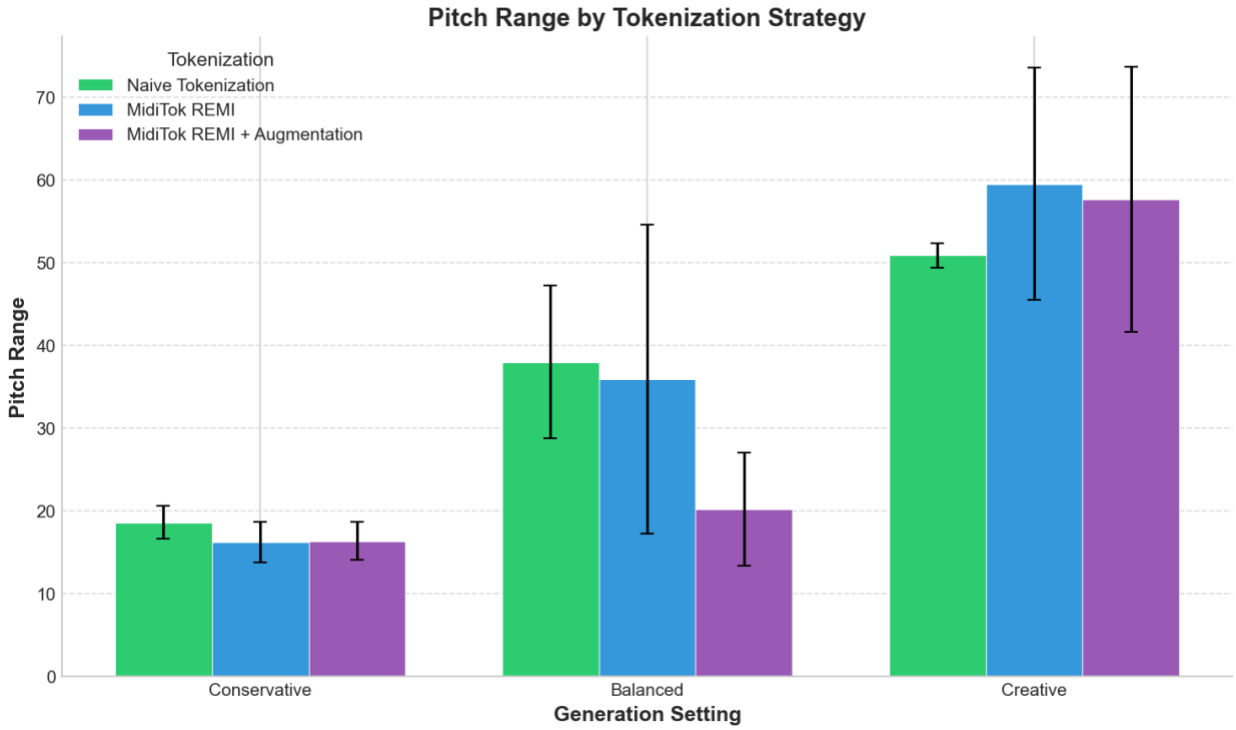
7. Experiment 3: Effect of Token Type on Generation

This experiment compared naïve, miditok, and augmented miditok LSTM models. 90 MIDI files were generated by each model and are evaluated. I expected naïve models to perform better than the standard miditok model, but I assumed that by augmenting the miditok model's dataset, it could generalize better and become comparable to the naïve model in performance.

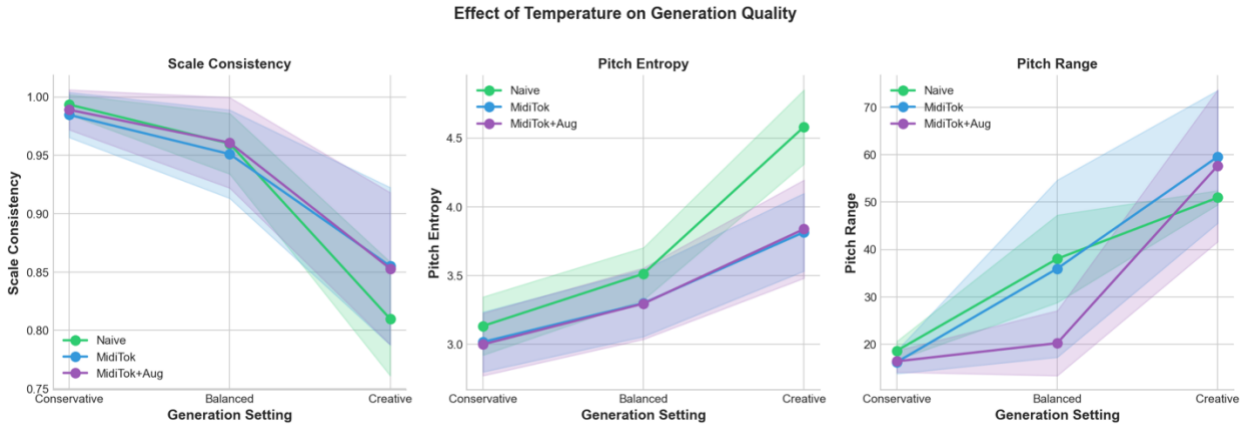
Each model generated 30 MIDI files with 3 types of generation arguments:

- Conservative: p = 0.85, Temp = 0.8
- Balanced: p = 0.9, Temp = 1.2
- Creative: p = 0.95, Temp = 1.2

7.1 Pitch Range by Tokenization Strategy



7.2 Effect of Temperature on Generation Quality



Findings:

- Naïve models had the highest pitch range in conservative and balanced settings.
- With creative sampling, both miditok models surpassed the naïve model in pitch range.
- The augmented miditok model consistently outperformed the standard miditok model across most metrics, showing that a larger dataset is necessary for more complicated tokenization methods.

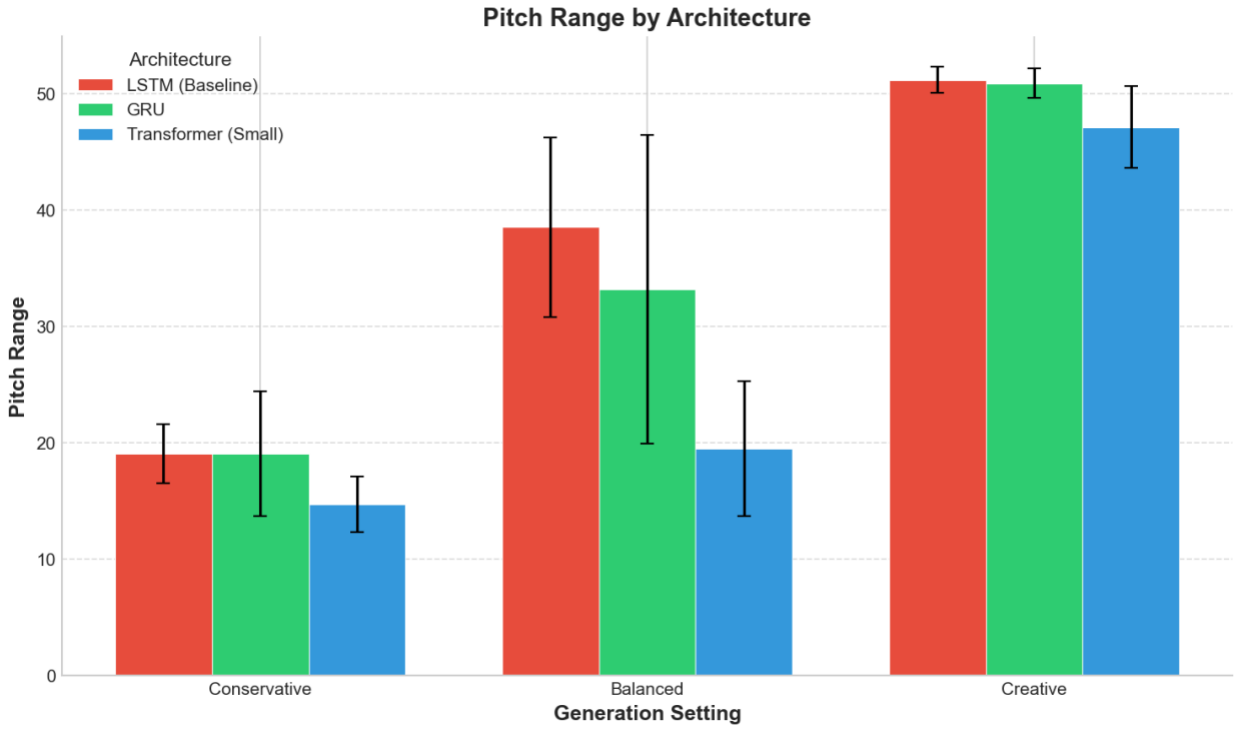
8. Effect of Model Architecture on Generation

The baseline LSTM model got compared to:

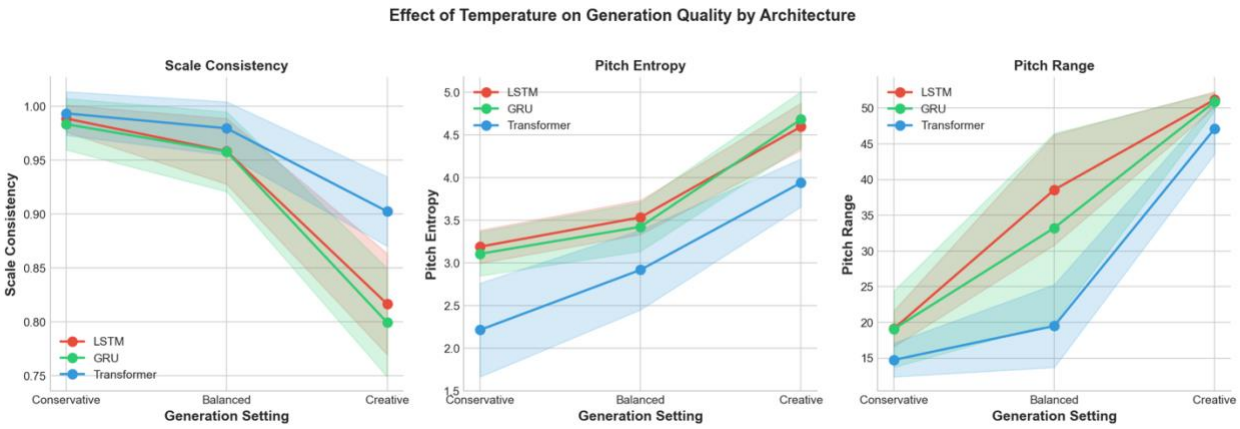
- A GRU model
- A small Transformer model

All models were trained under the same hyperparameters.
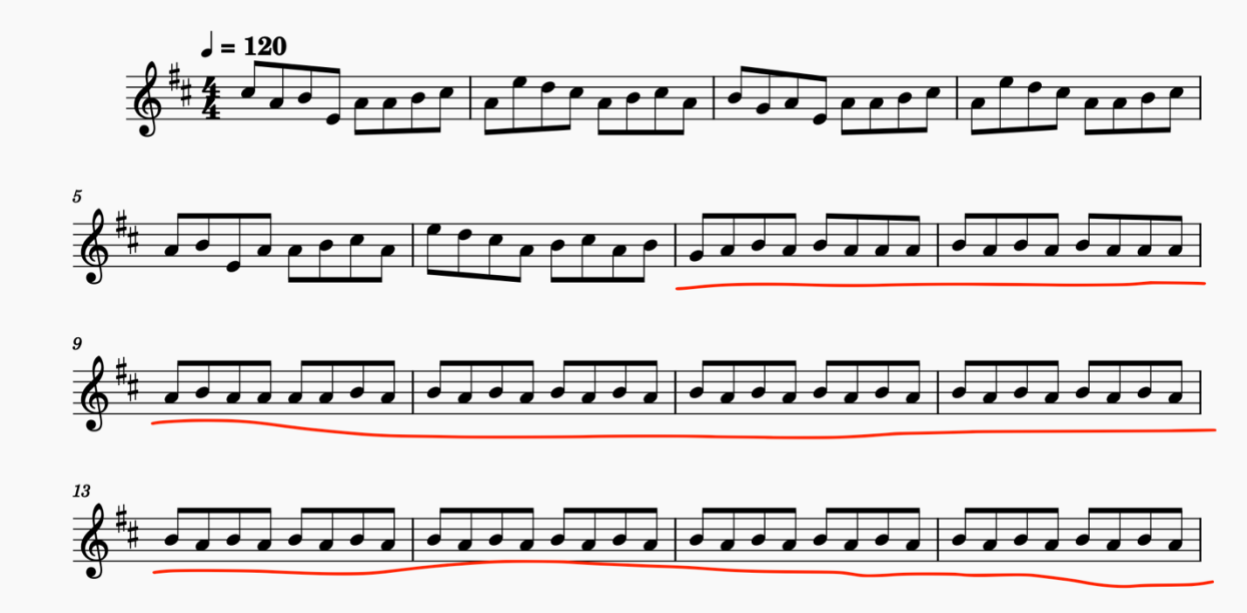
8.1 Pitch Range by Architecture



8.2 Effect of Temperature on Generation Quality by Architecture



8.3 Transformer Output With Top P Sampling Showing Repetitive Sequences



Findings:

- GRU performance was close to the LSTM but slightly worse.
- Transformer performance was consistently the worst. Even with top-p sampling, it produced repetitive sequences with low pitch variety.

This highlights that Transformer models require much larger datasets to produce useful and coherent music.

9. Experiment 5: Effect of Discriminator Architecture on Generation

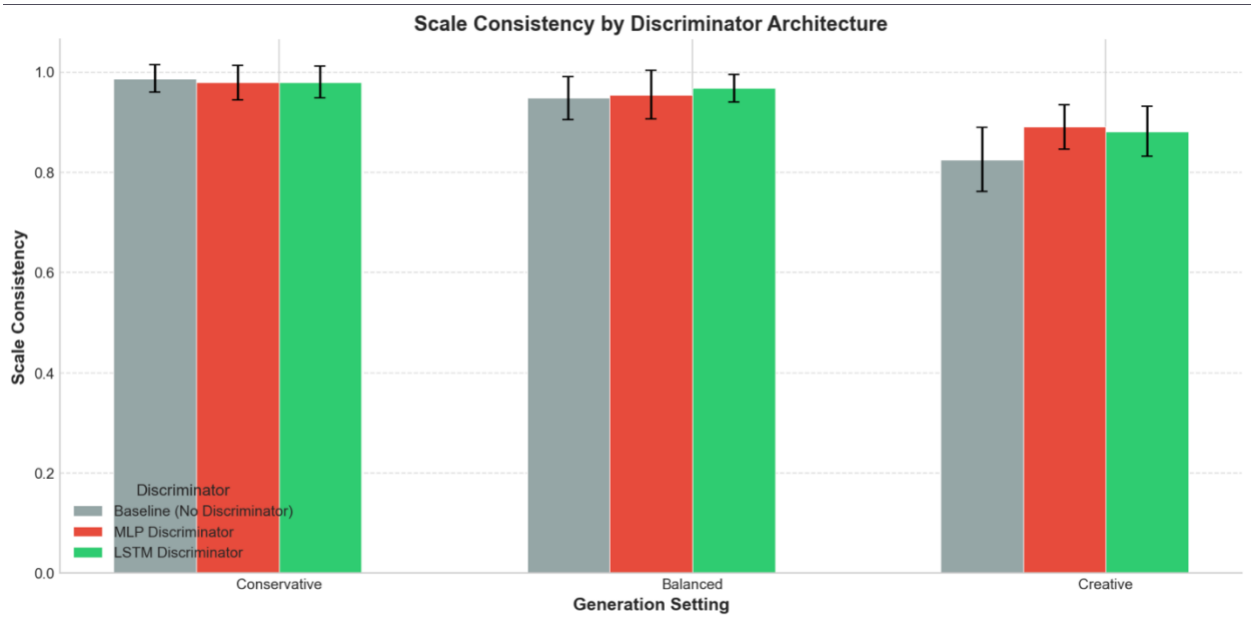
The baseline naïve LSTM model generated with:

- No discriminator guidance
- A MLP discriminator model
- A LSTM discriminator model

9.1 Distribution of Pitch Class Entropy by Discriminator



9.2 Scale Consistency by Discriminator Architecture



Findings:

- Scale consistency decreased with higher temperature, regardless of guidance.
- For conservative generation, having no guidance resulted in the lowest mean pitch class entropy (the guided models had more variation).
- Discriminator guidance during balanced and creative settings led to lower note variation than unguided.

10. Discussion

Across all the experiments the following seemed to be consistent:

1. Tokenization strongly influences generalization. Naïve tokens work much better than miditok for small datasets because they require less context.
2. Augmentation improves the performance of midtok models.
3. Higher creativity settings like temperature expand pitch range but can also lead to incoherent music.
4. Discriminator guidance improves correctness but reduces expressiveness. It filters out note selections that aren't harmonically "correct", leaving boring note selections at times.
5. Model architecture seems to matter less than tokenization and dataset size. Transformer performed the worst on the small dataset.

It appears that more complex architectures don't compensate for limited datasets when training MIDI-based models.

11. Conclusion

This project created a modular and flexible framework for experimenting with symbolic music generation. Through a series of controlled experiments, I analyzed how tokenization, sampling settings, discriminator guidance, and architecture affects musical metrics in generated MIDI files.

My key conclusions are:

- Naïve tokenization with LSTM models offer the best performance for small datasets (compared to the other tested configurations).
- Miditok models require larger datasets to generalize effectively.
- Discriminator guidance improves theoretical musical correctness, but may sound worse or less interesting.
- Transformers aren't suitable for small symbolic music datasets.

12. Sources

- https://medium.com/@anishnama20/understanding-gated-recurrent-unit-gru-in-deep-learning-2e54923f3e2
- Zheng Jiang (2019) Automatic Analysis of Music in Standard MIDI Files
- Zhu, Y., Baca, J., Rekabdar, B., Rawassizadeh, R. (2023). A Survey of AI Music Generation Tools and Models
- Briot, J., Hadjeres, G., Pachet, F. (2017). Deep Learning Techniques for Music Generation -- A Survey
- Bhandari, K., Roy, A., Wang, K., Puri, G., Colton, S., Herremans, D. (2024). Text2midi: Generating Symbolic Music from Captions
- Yang, L., Chou, S., Yang, Y. (2017). MidiNet: A Convolutional Generative Adversarial Network for Symbolic-domain Music Generation
- Tian, S., Zhang, C., Yuan, W., Tan, W., Zhu, W. (2025). XMusic: Towards a Generalized and Controllable Symbolic Music Generation Framework
- Colin Raffel. "Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching". _PhD Thesis_, 2016